project_summary

October 19, 2026

1 Model Discovery to Validated Inference

1.1 Goal

Build and validate a workflow that chooses **Hugging Face** models and identifies strong *serving* configurations for a specified GPU and workload. The workflow combines *hf-model-watch* for model discovery and evidence assessment with *llm-optimizer* for reproducible automated hyperparameter tuning across **vLLM, SGLang, and TensorRT-LLM**.

The immediate priority is **validating optimization on more models**. We want to establish whether improvements, sampler choices, and Pareto tradeoffs generalize across model sizes, architectures, serving engines, and concurrency levels.

1.2 Current progress

- Implemented framework-specific configuration search and benchmark adapters for vLLM, SGLang, and TensorRT-LLM.
- Implemented automated hyperparameter optimization to evaluate and improve LLM inference performance across multiple serving frameworks. We explore hardware and runtime-specific configurations, evaluate latency–throughput-ttft tradeoffs, and conduct reproducible GPU benchmarks to identify efficient deployment settings.
- Began Phase A/B work, including H100 SGLang, Vllm, trtllm experiments. These establish an implementation baseline, **not yet enough evidence of model-agnostic performance gains**.

1.3 Validation Workflow

1.3.1 1. Expand model coverage

Selecting models from multiple model families, parameter scales, and speculative decoding architectures to evaluate how optimization performance generalizes across different inference workloads. Include models with different speculative decoding heads where supported. Finalize model selections based on GPU memory requirements, framework compatibility, and support for the selected decoding methods.

1.3.2 2. Use comparable workloads

For each compatible model/engine pair, run fixed, token-controlled workloads:

Profile	Input tokens	Output tokens	Concurrency
Interactive	512	128	1
Concurrent serving	512	128	128
Long context	4,096	256	32

1.4 Requested GPU resources

Use	Allocation	Maximum usage
Parallel Nebius AI serverless jobs experiments	3 independent H100 GPUs, one per serving framework, up to 12 hours/day for 7 days	252 GPU-hours
Pipeline debugging and exploratory notebooks	1 H100 GPU, up to 12 hours/day for 7 days	84 GPU-hours
Total ceiling	**Up to 4 GPUs concurrently**	**336 GPU-hours**

Each worker runs a **single-GPU** model deployment; this does not require multi-node or tensor-parallel inference. Target approximately 16 vCPUs, 200 GB host RAM, and 530 GB SSD per experiment worker, subject to available cloud presets. A small persistent CPU controller and approximately 100 GB initial S3-compatible artifact storage support orchestration and results. GPUs are stopped when work completes; actual usage may be lower than the ceiling.

1.5 Seven-day schedule

Period	Primary work
Day 1	Verify worker images, drivers, engine/model compatibility, and baseline measurements; finalize additional models
Days 2–5	Run model-by-model Optuna studies with fixed workloads and bounded budgets
Day 5-6	Repeat shortlisted configurations, test robustness, and investigate failures
Day 7	Consolidate comparisons, identify generalizable findings, and publish reproducible configurations and reports

1.6 Goals

1. **Diverse model validation:** Different model architecture and size, supported engines, workloads, successes, and compatibility failures.
2. **Optimization report:** baseline versus tuned throughput, p95 E2E, and p95 TTFT; Pareto frontiers, sampler comparisons, trial cost, and variability.

3. **Reproducible artifacts:** selected engine configurations, container versions, seeds, benchmark manifests, logs, and checkpoints.
4. **Deployment guidance:** workload-specific recommended configurations, limits of generalization, and quality checks.

Success criterion: demonstrate independently repeatable tuning results on Different models, or clearly document where the optimizer fails to improve the baseline. No speedup is assumed in advance. All conclusions remain specific to the tested hardware, workloads, and runtime versions.